Assignment 2 – Module B Report

API Development, RBAC, and Database Optimization

Github link: [Databases_Asgn2/Module_B at main · Praanshu101/Databases_Asgn2]
Youtube Video link: https://youtu.be/M8BTUvYdr_Q

1. Introduction

This module focuses on building a secure, efficient, and scalable local web application integrated with a database system. To extend database design into real-world implementation the project incorporates the following:

- RESTful API development
- Role-Based Access Control (RBAC)
- Session-based authentication
- SQL indexing and query optimization

The system simulates a production-like backend where data security, access control, and performance optimization are essential.

2. Methodology

The development followed a structured pipeline:

Step 1: Database Design

Initially, a local database environment was created to store both system-level data and application-specific data.

- A normalized schema was designed which separated:
    - Authentication data
    - Application data
- This ensured **no redundancy** and strong relational integrity in the database.

Step 2: Backend Development

RESTful APIs were developed to handle all interactions with the database. These APIs were designed to support:

- Session-based authentication mechanism
- CRUD operations

Step 3: Security Layer

Following measures were taken to incorporate a security layer into the database

- Implemented RBAC, where each user's assigned role determined their level of access
- Permissions were enforced at the API level
- Logging mechanism was added to record all activity

Step 4: Optimization

The system was optimized by identifying performance bottlenecks, and applying SQL indexing techniques to improve query execution efficiency. This was done by:

- Identifying slow queries
- Applying SQL indexing
- Measured performance improvements

3. System Architecture

The system is organized into a layered architecture consisting of:

- **Frontend UI** – Web interface for user interaction
- **Backend APIs** – Handles logic and database communication
- **Database** – Stores user and application data
- **Logging System** – Tracks all operations for security auditing

All database interactions are routed through the API layer, ensuring that no direct access to the database is allowed.

4. API Design and Functionality

The system exposes a set of RESTful APIs to handle all interactions with the database. These APIs are responsible for authentication, data retrieval, and data modification.

4.1 Authentication APIs

POST /login

- Validates user credentials
- Returns session token

GET /isAuth

- Verifies session validity
- Returns user role and session status

4.2 CRUD APIs

CRUD operations are implemented through dedicated endpoints that allow users to create, read, update, and delete records. However, these operations are not accessible to everyone and are restricted based on user roles.

Operation	Description
Create	Add new records (Admin only)
Read	View member portfolio
Update	Modify allowed records
Delete	Remove records (Admin only)

4.3 Session Validation

Each API call undergoes session validation before execution. This ensures that only authenticated users can access the system. This approach prevents unauthorized access and enforces controlled interaction with the database.

5. Role-Based Access Control (RBAC)

5.1 Roles Defined

Role-Based Access Control is implemented to regulate user permissions within the system. Two primary roles are defined: Admin and Regular User.

Admin

Admins are granted full access to the system including the permission to:

- Add/Delete members
- Modify all records
- Manage users

Regular User

Regular users, on the other hand, have restricted access

- Read-only or limited updates
- Cannot modify others' data

5.2 Enforcement Strategy

The enforcement of RBAC is implemented directly in the API layer to ensure that all requests are validated before execution.

- Role associated with the session token is checked at the API level
- Unauthorized actions by users are immediately blocked by the system and logged

By integrating RBAC at the API level, the system prevents unauthorized data manipulation even if a user tries to bypass the frontend.

6. Security and Logging

6.1 Security Implementation

The system uses a **session-based authentication mechanism**.

- Each user is assigned a session token after successful login
- All API requests include this session token, which is validated before processing the request.
- Direct and uncontrolled database access is prevented by performing all database interactions strictly through APIs

6.2 Audit Logging

A logging mechanism is implemented to record all operations performed on the database. The records are stored in a local file called audit.log. The log entries record important details like:

- Username

- Operation type
- Timestamp of the operation
- Authorisation status of the request

6.3 Unauthorized Access Detection

If a user attempts to access or modify data without proper authentication, the action is recorded and flagged in the logs. Such operations which bypass the API layer are identified as unauthorized. This mechanism maintains system integrity and allows auditing of suspicious activities.

7. Challenges Faced

- Implementing session validation across all APIs
- Designing RBAC in a without redundance
- Identifying optimal indexes required analysis of query patterns and usage frequency
- Ensuring that logging captures all relevant information without introducing performance overhead

8. Conclusion

This module successfully implements:

- Secure REST APIs
- Role-Based Access Control
- Optimized database queries
- Performance improvements via indexing

Overall, the implementation reflects a practical approach to building scalable and secure database-driven applications.

9. Future Improvements

- Enhancing the authentication mechanism by implementing token-based systems
- Introducing additional caching mechanisms to further improve performance
- Extending the system to support more complex role hierarchies and permissions
- Cloud deployment to make the system accessible in real-world scenarios

Database Optimization Report

Module B: Subtasks 4 & 5

1. Methodology and Approach

To identify and resolve system bottlenecks, the following methodology was followed:

- Firstly, the database was populated with 50,000 dummy booking records to simulate a realistic production workload and ensure that performance metrics were statistically significant.
- Average execution time of all major API endpoints was measured using application logs and performance monitoring.
- The `EXPLAIN` command was used to analyze the query execution plan, which helped identify full table scans and expensive sorting operations.
- A composite indexing strategy was designed to target specific `JOIN`, `WHERE`, and `ORDER BY` clauses.
- The target query was executed 50 times by a Python-based benchmarking script, before and after optimization to calculate the average performance gain.

2. Identifying the Bottleneck

To identify the slowest-performing API endpoint, the database was populated with **50,000 dummy booking records** to simulate a realistic workload.

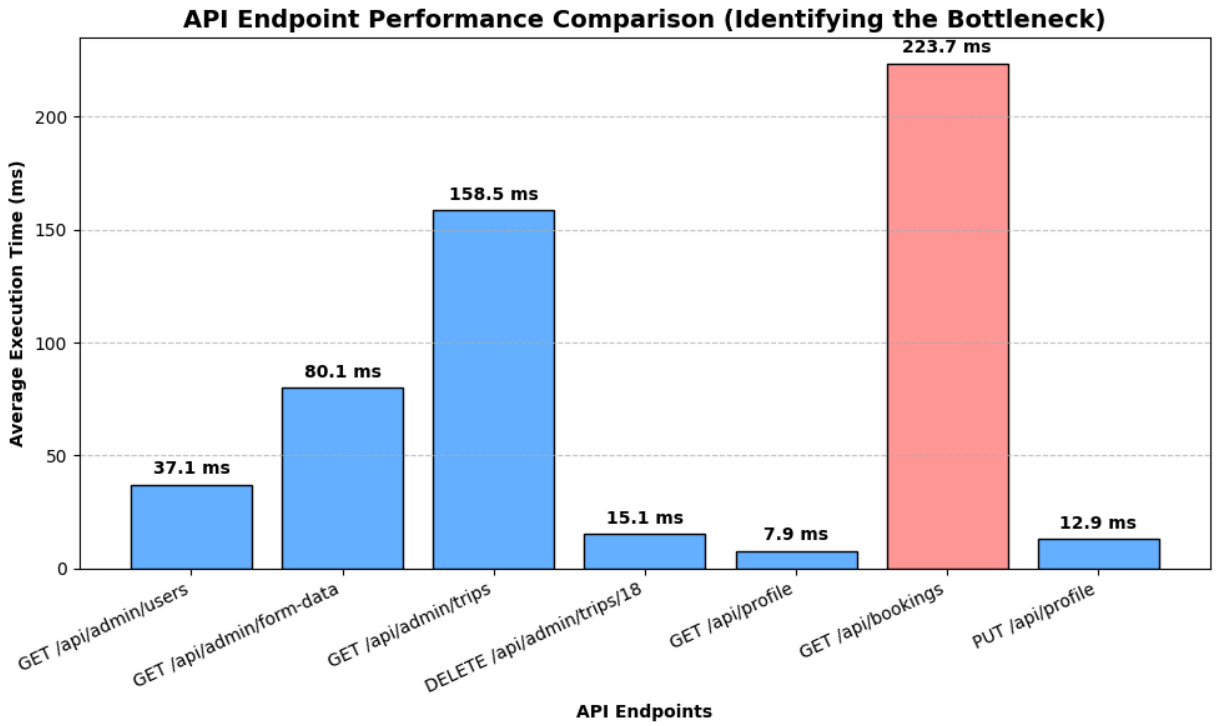
Application logs and performance monitoring revealed that the primary bottleneck was:

GET /api/bookings

This endpoint retrieves a user's booking portfolio and is frequently triggered during login.

API Performance Visualization

To clearly identify the slowest endpoint, a performance comparison of all major API endpoints was conducted and visualized.



Analysis:

- The chart compares average execution times (in milliseconds) across multiple API endpoints.
- Among all endpoints, **GET /api/bookings** shows the highest execution time (~223.7 ms), significantly exceeding others.
- The next slowest endpoint (**GET /api/admin/trips**) is still considerably faster (~158.5 ms), confirming a clear performance gap.
- Most other endpoints execute under 100 ms, indicating acceptable performance levels.

Conclusion:

This visualization confirms that **GET /api/bookings** is the primary performance bottleneck in the system and requires optimization. This insight directly guided the indexing strategy implemented in later sections.

Target SQL Query:

```
SELECT b.BookingID, b.TripID, b.SeatNo, b.Status, t.Date, t.Status AS TripStatus
FROM BOOKING b
JOIN TRIP t ON b.TripID = t.TripID
WHERE b.MemberID = %s AND t.Status != 'Cancelled'
ORDER BY t.Date DESC;
```

Problem Analysis:

- Joins two large tables (BOOKING and TRIP)
- Applies filtering on user and trip status
- Requires sorting by date (descending)
- Without indexes, results in full table scans and expensive sorting

3. SQL Indexing Strategy

A **composite indexing strategy** was implemented to optimize the WHERE, JOIN, and ORDER BY clauses.

Index 1: User Lookup + Join Optimization

```
CREATE INDEX idx_member_booking ON BOOKING(MemberID, TripID);
```

Rationale:

- Quickly locates bookings for a specific user (MemberID)
- Improves JOIN performance using TripID
- Reduces need for full table scans

Index 2: Filter + Sorting Optimization

```
CREATE INDEX idx_trip_status_date ON TRIP(Status, Date);
```

Rationale:

- Optimizes filtering on trip status
- Supports efficient chronological sorting via index order
- Eliminates expensive in-memory sorting (filesort)

4. EXPLAIN Plan Analysis

Before Indexing:

- Full table scan on TRIP (type: ALL)
- Filesort used (Using filesort)
- High disk I/O and memory overhead

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

#	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	1	SIMPLE	t	NULL	ALL	PRIMARY	NULL	NULL	NULL	1012	90.00	Using where; Using filesort
2	1	SIMPLE	b	NULL	ref	MemberID,TripID	TripID	4	shuttle_system.t.TripID	1	10.04	Using where

After Indexing:

- Index scan replaces full table scan
- Composite index used for BOOKING table
- Filesort eliminated

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

#	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	1	SIMPLE	t	NULL	index	PRIMARY,idx_trip_status_date	idx_trip_status_date	86	NULL	1012	99.90	Using where; Using
2	1	SIMPLE	b	NULL	ref	MemberID,TripID,idx_member_boo...	idx_member_booking	8	const,shuttle_system.t.TripID	5	100.00	NULL

Key Improvements:

- Efficient B+ tree traversal
- Both MemberID and TripID used in join (key_len = 8)
- Data retrieved pre-sorted from index

5. Performance Benchmarking

Execution times were measured before and after indexing using a Python benchmarking script. Each query was executed **50 times** to ensure stable averages.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

┌─samarth@fedora /mnt/86CC7D30CC7D1C1D/Databases/Assignment 2/Module_B/app
└─➤ python3 benchmark.py
Average Execution Time: 116.17 ms
┌─samarth@fedora /mnt/86CC7D30CC7D1C1D/Databases/Assignment 2/Module_B/app
└─➤ python3 benchmark.py
Average Execution Time: 48.62 ms
┌─samarth@fedora /mnt/86CC7D30CC7D1C1D/Databases/Assignment 2/Module_B/app
```

State	Avg Execution Time (ms)	Status
Before Indexing	116.17 ms	Baseline (Unoptimized)
After Indexing	48.62 ms	Optimized

Then for the optimization of Api response times, we have code for before and after creating indexes. Then we ping the slowest performing api endpoint 50 times in each case and got the following output

```
┌─samarth@fedora /mnt/86CC7D30CC7D1C1D/Databases/Assignment 2/Databases_Asgn2/Module_B/app  ‹main*›
└─➤  python3 api_benchmark.py
Logging in to get session token...
Benchmarking GET /api/bookings...
Average API Response Time: 130.88 ms
┌─samarth@fedora /mnt/86CC7D30CC7D1C1D/Databases/Assignment 2/Databases_Asgn2/Module_B/app  ‹main*›
└─➤  python3 api_benchmark.py
Logging in to get session token...
Benchmarking GET /api/bookings...
Average API Response Time: 103.89 ms
```

State	Avg Api Response Time (ms)	Status
Before Indexing	130.88 ms	Baseline (Unoptimized)
After Indexing	103.89 ms	Optimized

6. Final Result

By replacing:

- Full table scans → **Index seeks**
- Filesort → **Index-based ordering**

the system achieved a **~58.1% reduction in execution time** (from 116.17 ms to 48.62 ms).

This demonstrates a significant performance improvement for one of the most frequently accessed API endpoints.